

SDA – Seminar 2 – Complexități

Cuprins:

- Introducere
- Notății asimptotice de complexitate
- Exerciții

Introducere

Eficiența unui algoritm este determinată de cantitatea de resurse pe care le consumă, în termeni de timp și memorie.

Ca modalități de măsurare a complexității timp, avem:

- Analiza **empirică** (sau experimentală), care constă în măsurarea timpului efectiv de execuție (acesta este un avantaj al metodei), pentru un subset al datelor de intrare posibile, fără a putea prezice performanța pentru toate datele de intrare posibile (acesta este un dezavantaj al metodei). Timpul de execuție va fi exprimat numeric, ca număr efectiv de secunde necesare procesării.
- Analiza **asimptotică** (sau matematică) este analiza în care surprindem nu timpii exacti de execuție (acesta este un dezavantaj al metodei), ci ordinul de creștere al timpului de execuție, pentru toate datele de intrare posibile (acesta este un avantaj al metodei).

Complexitatea unui algoritm poate să depindă și de valorile datelor de intrare, nu doar de dimensiunea lor. Prin urmare, distingem următoarele tipuri de analiză de complexitate:

- Analiza complexității în cazul **defavorabil**, adică pentru date de intrare defavorabile (care implică număr maxim de operații). Această analiză este importantă deoarece oferă o garanție, aceea că algoritmul nu poate fi mai inefficient, indiferent de valorile datelor de intrare.
- Analiza complexității în cazul **favorabil**, adică pentru date de intrare favorabile (care implică număr minim de operații), potrivită, mai degrabă, pentru probleme în care datele de intrare tind să fie favorabile.
- Analiza complexității în cazul **mediu**, adică pentru date de intrare **aleatorii**. O astfel de analiză este în mod particular utilă, însă este, totodată,

mai dificil de realizat. De ce? Deoarece necesită cunoașterea (altfel, presupunerea) unei distribuții statistice a valorilor datelor de intrare pentru a calcula media ponderată cu probabilitățile lor de apariție.

Notății asimptotice de complexitate

Pentru clase de complexitate avem următoarele notații asimptotice: O , Ω , Θ .

Definiții:

- **$T(n) \in O(f(n))$** $\Leftrightarrow \exists$ constantele $c \in \mathbb{R}_+$, $c > 0$ și $n_0 \in \mathbb{N}$ a.i. $0 \leq T(n) \leq c * f(n)$ pentru orice $n \geq n_0$
- **$T(n) \in O(f(n))$** dacă $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = 0$ sau o constantă nenulă

Semnificație: pentru valori (suficient de) mari ale dimensiunii datelor de intrare, $c * f(n)$ este o **limită superioară** pentru $T(n)$

- **$T(n) \in \Omega(f(n))$** $\Leftrightarrow \exists$ constantele $c \in \mathbb{R}_+$, $c > 0$ și $n_0 \in \mathbb{N}$ a.i. $0 \leq c * f(n) \leq T(n)$ pentru orice $n \geq n_0$
- **$T(n) \in \Omega(f(n))$** dacă $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = \infty$ sau o constantă nenulă

Semnificație: pentru valori (suficient de) mari ale dimensiunii intrării, $c * f(n)$ este o **limită inferioară** pentru $T(n)$

- **$T(n) \in \Theta(f(n))$** $\Leftrightarrow \exists$ constantele $c \in \mathbb{R}_+$, $c_1, c_2 > 0$ și $n_0 \in \mathbb{N}$ a.i. $0 \leq c_1 * f(n) \leq T(n) \leq c_2 * f(n)$ pentru orice $n \geq n_0$
- **$T(n) \in \Theta(f(n))$** dacă $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)}$ este o constantă nenulă

Semnificație: pentru valori (suficient de) mari ale dimensiunii intrării, $c_1 * f(n)$ este o **limită inferioară** pentru $T(n)$, iar $c_2 * f(n)$ este o **limită superioară**

Complexitățile algoritmilor cunoscuți de căutare și sortare:

Algoritm	Complexitate Timp				Complexitate Spațiu (extra)
	CF	CD	CM	Total	
Căutare secvențială	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(1)$
Căutare binară	$\Theta(1)$	$\Theta(\log_2 n)$	$\Theta(\log_2 n)$	$O(\log_2 n)$	$\Theta(1)$
Sortare prin selecție	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$ - in place
Sortare prin inserție	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(n^2)$	$\Theta(1)$ - in place
Sortare prin metoda bulelor	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(n^2)$	$\Theta(1)$ - in place
Quick Sort	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(n \log_2 n)$	$O(n^2)$	$\Theta(1)$ - in place
Sortare prin interclasare (Merge Sort)	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n)$ - out of place

Algoritmii de sortare *in place* sortează șirul fără a folosi structuri de date suplimentare pentru variabilele auxiliare (doar spațiu de memorie adițional constant). De exemplu, un algoritm de sortare care efectuează ordonarea doar prin interschimbări de elemente este *in place*. Un algoritm care nu este in place este *out of place*.

1. Adevărat sau Fals?

- $n^2 \in O(n^3)$ - Adevărat
- $n^3 \in O(n^2)$ - Fals
- $2^{n+1} \in \Theta(2^n)$ - Adevărat
- $n^2 \in \Theta(n^3)$ - Fals
- $2^n \in O(n!)$ - Adevărat
- $\log_{10} n \in \Theta(\log_2 n)$ - Adevărat
- $O(n) + \Theta(n^2) = \Theta(n^2)$ - Adevărat
- $\Theta(n) + O(n^2) = O(n^2)$ - Adevărat
- $O(n) + O(n^2) = O(n^2)$ - Adevărat
- $O(f) + O(g) = O(\max\{f, g\})$ - Adevărat
- $O(n) + \Theta(n) = O(n)$ - Adevărat, dar $\Theta(n)$ este mai exact

2. Construiți un algoritm având complexitatea timp $\Theta(n \log_2 n)$.

Exemplu de soluție:

```

pentru  $i \leftarrow 1, n$  execută
     $j \leftarrow n$ 
    cât timp  $j \neq 0$  execută
         $j \leftarrow \left\lfloor \frac{j}{2} \right\rfloor$ 
    sf_cât timp
sf_pentru

```

Un alt exemplu de soluție :

```

pentru  $i \leftarrow 1, n$  execută
     $j \leftarrow i$ 
    cât timp  $j \neq 0$  execută
         $j \leftarrow \left\lfloor \frac{j}{2} \right\rfloor$ 
    sf_cât timp
sf_pentru

```

Complexitatea acestui algoritm este: $T(n) = \sum_{i=1}^n \log_2 i = \log_2(n!)$

Vom demonstra că $T(n) \in \Theta(n \log_2 n)$.

Putem obține o margine superioară astfel:

$$T(n) = \sum_{i=1}^n \log_2 i \leq \sum_{i=1}^n \log_2 n = n \log_2 n \quad (1)$$

De asemenea, putem obține o margine inferioară pentru $T(n)$ astfel

$$T(n) = \sum_{i=1}^n \log_2 i \geq \sum_{i=n/2}^n \log_2 i \geq \sum_{i=n/2}^n \log_2 \frac{n}{2} = \frac{n}{2} \log_2 \frac{n}{2} = \frac{n}{2} \log_2 n - \frac{n}{2} \geq \frac{n}{4} \log_2 n, \forall n \geq 4 \quad (2)$$

Din (1) și (2) $\Rightarrow T(n) \in \Theta(n \log_2 n)$

O metodă alternativă de demonstrație folosește aproximarea lui Stirling.

3. Calculați complexitatea timp pentru următorii 2 algoritmi:

a)

```

gasit  $\leftarrow$  fals
pentru  $i \leftarrow 1, n$  execută
    dacă  $x_i = a$  atunci
        gasit  $\leftarrow$  adevarat
    sf_dacă
sf_pentru

```

Soluție:

Se observă că ciclul se execută independent de valoarea variabilei *găsit*.

$$\left. \begin{array}{l} CF: \theta(n) \\ CD: \theta(n) \end{array} \right\} = \Theta(n)$$

b)

```
gasit ← fals
i ← 1
cât timp gasit = false și i ≤ n execută
    dacă xi = a atunci
        gasit ← adevarat
    sf_dacă
    i ← i + 1
sf_cât timp
```

CF: $\Theta(1)$

CD: $\Theta(n)$

CM: sunt n+1 cazuri posibile (elementul se găsește pe oricare dintre cele n poziții sau pe niciuna). Presupunem că toate aceste cazuri au probabilitate egală.

$$T(n) = \sum_{I \in D} P(I) * E(I) = \frac{1}{n+1} + \frac{2}{n+1} + \dots + \frac{n}{n+1} + \frac{n}{n+1} = \frac{n * (n+1)}{2 * (n+1)} + \frac{n}{n+1} \in \Theta(n)$$

Complexitate totală: $O(n)$

4. Fie X este un șir de n numere naturale, fiecare element fiind $\leq n$. Se dă următorul algoritm.

```
k ← 0
pentru i ← 1, n execută
    pentru j ← 1, xi execută
        (*) k ← k + xj
    sf_pentru
sf_pentru
```

Observând că operația (*) se efectuează de un număr de ori egal cu suma elementelor șirului, este corect să determinăm și să exprimăm complexitatea timp precum mai jos ?

$$T(n) = \sum_{i=1}^n \sum_{j=1}^{x_i} 1 = \sum_{i=1}^n x_i = s \text{ (suma elementelor)} \in \Theta(s)$$

Exemplificăm un șir care respectă proprietatea enunțată:

$$\text{Fie } x_i = \begin{cases} 1, & \text{dacă } i \text{ este pătrat perfect} \\ 0, & \text{altfel} \end{cases}$$

Deducem că suma elementelor șirului este $s = \lfloor \sqrt{n} \rfloor$, deci, conform calculului de mai sus, complexitatea este $\Theta(\sqrt{n})$, deci este sub-liniară. Dar ciclul exterior efectuează n iterații. Survine, astfel, o contradicție.

Exprimarea corectă a complexității este:

$$T(n) = \sum_{i=1}^n (1 + x_i) = s + n \in \Theta(\max\{n, s\}) = \Theta(n + s)$$

Obs: Dacă elementele șirului ar fi fost numere naturale nenule, exprimarea inițială a complexității ar fi fost corectă

5. a) Determinați dacă un șir arbitrar de numere conține cel puțin două elemente egale folosind un algoritm de complexitate $\Theta(n \log(n))$.
- b) Se dă un număr k și un șir arbitrar de numere $x_1 \dots x_n$. Determinați dacă există două numere în șir care au suma k . Care este complexitatea soluției?
- c) Se dă un șir ordonat x format din elemente întregi distincte. Determinați dacă există un indice i astfel încât $x_i = i$. Care este complexitatea soluției?

Soluție:

a) Se va aplica MergeSort pe șir, rezolvarea rezumându-se ulterior la parcurgerea șirului și verificarea existenței a două elemente egale, aflate pe poziții consecutive (într-un vector sortat elementele egale se pot afla doar pe poziții consecutive). Această parcurgere se efectuează în $O(n)$. Complexitatea algoritmului: $\Theta(n \log(n)) + O(n) = \Theta(n \log(n))$

b) Metoda 1: O soluție posibilă este verificarea tuturor perechilor de numere din șir (x_i, x_j) , $i < j$ și compararea sumei lor cu valoarea k . Căutarea se oprește dacă găsim o pereche având proprietatea cerută. Această abordare are complexitatea $O(n^2)$.

Metoda 2: O altă soluție poate fi obținută pornind de la observația că în metoda 1, pentru fiecare i , căutăm elementul $k - x_i$ în șir. Căutarea poate fi optimizată (prin folosirea căutării binare) dacă pentru început sortăm vectorul. Această sortare se poate efectua folosind Merge Sort. Ulterior, pentru fiecare element x_i , vom aplica un

algoritm de căutare binară pentru valoarea $k - x_i$. În plus, poziția j cu proprietatea că $x_j = k - x_i$ trebuie să fie diferită de i . Ne putem asigura de acest lucru căutând de fiecare dată valoarea $k - x_i$ nu în tot vectorul x , ci în subsecvența $x_{i+1}, \dots, x_n$.

Complexitatea rezolvării va fi: $\Theta(n \log_2 n) + O(n \log_2 n) = \Theta(n \log_2 n)$

Metoda 3: O a treia metodă de rezolvare va conține tot două etape. Prima va fi reprezentată de sortarea vectorului în ordine crescătoare. Vom folosi doi indici: i și j inițializați la capetele șirului ($i \leftarrow 1, j \leftarrow n$).

Dacă $x_1 + x_n = k$, am găsit o pereche de elemente care respectă cerința.

Dacă $x_1 + x_n > k$, avem nevoie de un element mai mic pentru a obține suma $k \Rightarrow$ vom decrementa j ($k < x_1 + x_n \leq x_2 + x_n \leq x_3 + x_n \leq \dots \leq x_{n-1} + x_n \Rightarrow x_n$ nu poate face parte dintr-o soluție) și vom continua procesul comparând suma k cu $x_i + x_j$.

Dacă $x_1 + x_n < k \Rightarrow$ vom incrementa i .

Căutarea se oprește dacă am găsit o pereche având suma cerută sau $i = j$. Această a doua etapă va avea complexitatea $O(n)$ – în cel mai rău caz vom parcurge întreg șirul. Întregul algoritm are complexitatea timp $\Theta(n \log_2 n) + O(n) = \Theta(n \log_2 n)$

Vom discuta o altă metodă de a rezolva această problemă într-un seminar viitor.

c) Vom folosi o idee asemănătoare cu ideea căutării binare. Calculăm mereu mijlocul intervalului $m = \lfloor (s+d)/2 \rfloor$. Dacă $x_m = m$, am identificat o soluție. Dacă $x_m > m$, fiindcă numerele sunt întregi $\Rightarrow x_m \geq m+1$; cum $x_{m+1} > x_m \Rightarrow x_{m+1} > m+1$. Se poate demonstra că $x_i > i$ pentru orice $i \geq m$. Astfel, nu vor exista soluții în a doua jumătate a șirului $\Rightarrow$ vom continua căutarea în prima jumătate. Similar, dacă $x_m < m$, vom continua căutarea în a doua jumătate a șirului. Complexitatea acestei soluții este $O(\log_2 n)$

6. Calculați complexitatea timp a următorului algoritm:

```

pentru  $i \leftarrow 1, n$  execută
    @ operatie elementara
sf_pentru
 $i \leftarrow 1$ 
 $k \leftarrow \text{adevarat}$ 
cât timp  $i \leq n - 1$  și  $k$  execută
     $j \leftarrow i$ 
     $k_1 \leftarrow \text{adevarat}$ 
    cât timp  $j \leq n$  și  $k_1$  execută
        @ operatie elementara ( $k_1$  poate fi modificat)
         $j \leftarrow j + 1$ 
    sf_cât timp
     $i \leftarrow i + 1$ 
    @ operatie elementara ( $k$  poate fi modificat)
sf_cât timp

```

Soluție:

CF: k, k_1 pot deveni *fals* după primul pas $\Rightarrow \Theta(n)$ (din cauza primului "*pentru*")

CD: k, k_1 nu devin *fals* niciodată

$$T(n) = n + \sum_{i=1}^{n-1} \sum_{j=i}^n 1 = n + \sum_{i=1}^{n-1} (n-i+1) = n + \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i + \sum_{i=1}^{n-1} 1 = n + n(n-1) - \frac{n(n-1)}{2} + n - 1 \in \Theta(n^2)$$

CM:

Considerăm pentru început ciclul *cât timp* interior. Pentru un i fixat, k_1 poate deveni *fals* după 1, 2, ..., $n-i+1$ iterații $\Rightarrow$ fiecare scenariu cu probabilitate: $\frac{1}{n-i+1}$

Astfel, numărul mediu de pași executați de ciclul *cât timp* interior este

$$T_{\text{interior}}(n, i) = \frac{1}{n-i+1} + \frac{2}{n-i+1} + \dots + \frac{n-i+1}{n-i+1} = \frac{(n-i+1)(n-i+2)}{2(n-i+1)} = \frac{n-i+2}{2}$$

Analizând ciclul *cât timp* exterior, observăm că variabila k poate deveni *fals* după 1, 2, ..., $n-1$ iterații, fiecare scenariu având probabilitatea $\frac{1}{n-1}$

Presupunând că ciclul *cât timp* exterior efectuează p iterații, vom calcula numărul de pași efectuați (când i este egal cu 1, se efectuează întâi un număr constant de pași, corespunzător atribuirilor, urmat de $T_{\text{interior}}(n, 1)$ pași, reprezentând numărul mediu de pași ai *cât timp*-ului interior; ulterior i se incrementează la 2, se efectuează $1 + T_{\text{interior}}(n, 2) \dots$) :

$$(1 + T_{\text{interior}}(n, 1)) + (1 + T_{\text{interior}}(n, 2)) + \dots + (1 + T_{\text{interior}}(n, p)) = p + \sum_{i=1}^p \frac{n-i+2}{2} = \frac{p(2n+7) - p^2}{4}$$

Astfel, numărul mediu de pași executați de ciclul *cât timp* exterior va fi:

$$\begin{aligned} \sum_{p=1}^{n-1} \frac{1}{n-1} * \frac{p(2n+7) - p^2}{4} &= \frac{1}{4(n-1)} \left((2n+7) \sum_{p=1}^{n-1} p - \sum_{p=1}^{n-1} p^2 \right) = \frac{1}{4(n-1)} \left(\frac{(2n+7)(n-1)n}{2} - \frac{(n-1)n(2n-1)}{6} \right) = \dots \\ \dots &= \frac{(3n(2n+7) - n(2n-1))}{24} = \frac{2n^2 + 11n}{12} \in \Theta(n^2) \end{aligned}$$

Pentru a calcula complexitatea medie a întregului algoritm, trebuie să adunăm și cei n pași efectuați de ciclul *pentru*, dar aceștia nu vor schimba clasa de complexitate.

Întrucât $CF = \Theta(n)$, $CM = CD = \Theta(n^2)$, complexitate totală este $O(n^2)$

7. Calculați complexitatea următorului algoritm:

```

subalgoritm p(x,s,d) este:
    daca s < d atunci
        m ← [(s+d)/2]
        pentru i ← s, d-1, executa
            @operatie elementara
        sf_pentru
        pentru i ← 1,2 executa
            p(x, s, m)
        sf_pentru
    sf_daca
sf_subalgoritm

```

apel: p(x, 1, n)

Solutie:

$$T(n) = \begin{cases} 2 * T\left(\frac{n}{2}\right) + n, & \text{daca } n > 1 \\ 1, & \text{altfel} \end{cases}$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Presupunem: $n = 2^k$

$$T(2^k) = 2 * T(2^{k-1}) + 2^k$$

$$2 * T(2^{k-1}) = 2^2 * T(2^{k-2}) + 2^k$$

$$2^2 * T(2^{k-2}) = 2^3 * T(2^{k-3}) + 2^k$$

$$\dots$$

$$2^{k-1} * T(2) = 2^k * T(1) + 2^k$$

$$T(2^k) = 2^k * T(1) + k * 2^k = k * 2^k + 2^k = n * \log_2 n + n \rightarrow T(n) \in \Theta(n \log_2 n)$$

8. Calculați complexitatea timp a următorului algoritm:

s ← 0

```

pentru i ← 1, n2 executa
    j ← i
    cattimp j ≠ 0 executa
        s ← s + j
        j ← j - 1
    sf_cattimp
sf_pentru

```

Soluție:

$$T(n) = \sum_{i=1}^{n^2} \sum_{j=1}^i 1 = \sum_{i=1}^{n^2} i = \frac{n^2 * (n^2 + 1)}{2} \in \Theta(n^4)$$

9. Calculați complexitatea timp a următorului algoritm:

```

s ← 0
pentru i ← 1, n2 executa
    j ← n
    cattimp j ≠ 0 executa
        s ← s + j - 10 * [j/10]
        j ← [j/10]
    sf_cattimp
sf_pentru

```

Soluție:

- Ciclul Cât timp se execută de $\log_{10} n$ ori.
- $T(n) \in \Theta(n^2 \log_{10} n) \Rightarrow T(n) \in \Theta(n^2 \log_2 n)$

10. Calculați complexitatea timp a următorului algoritm:

```

subalgoritm operație(n, i) este:
    daca n > 1 atunci
        i ← 2 * i
        m ← [n/2]
        operație(m, i-2)
        operație(m, i-1)
        operație(m, i+2)
        operație(m, i+1)
    altfel
        scrie i
    sf_daca
sf_subalg

```

Solutie:

$$T(n) = \begin{cases} 4 * T\left(\frac{n}{2}\right) + 1, & \text{dacă } n > 1 \\ 1, & \text{altfel} \end{cases}$$

Presupunem: $n = 2^k$

$$T(2^k) = 4 * T(2^{k-1}) + 1$$

$$4 * T(2^{k-1}) = 4^2 * T(2^{k-2}) + 4$$

$$4^2 * T(2^{k-2}) = 4^3 * T(2^{k-3}) + 4^2$$

$$\vdots$$
$$4^{k-1} * T(2) = 4^k * T(1) + 4^{k-1}$$

$$T(2^k) = 1 + 4 + \dots + 4^{k-1} + 4^k * T(1) = \frac{4^{k+1} - 1}{3} = \frac{4 * n^2 - 1}{3} \in \Theta(n^2)$$